

MVLU COLLEGE
A.I. Practical No.3

Aim:-To implement the Decision Tree Learning algorithm using the IPL 2018 Most Runs dataset, classify players into High and Low performance categories, evaluate the accuracy of the model on test data, and visualize the generated decision tree.

Description:-The program implements a Decision Tree Classification algorithm using the IPL 2018 Most Runs dataset. The dataset contains batting statistics of IPL players such as matches, innings, runs, average, strike rate, boundaries, and centuries. The program classifies players into High Performance and Low Performance categories based on the median number of runs. It then evaluates the model using accuracy, classification report, confusion matrix, and visualizes the generated Decision Tree

Code / Function Description Table

Step	Code / Function	Description
1	<code>import pandas as pd</code>	Imports Pandas for reading and processing the CSV dataset.
2	<code>import matplotlib.pyplot as plt</code>	Imports Matplotlib for visualizing the Decision Tree.
3	<code>from sklearn.model_selection import train_test_split</code>	Splits the dataset into training and testing data.
4	<code>from sklearn.tree import DecisionTreeClassifier</code>	Imports the Decision Tree classification algorithm.
5	<code>from sklearn.tree import plot_tree</code>	Used to generate the graphical representation of the Decision Tree.
6	<code>from sklearn.metrics import accuracy_score</code>	Calculates the accuracy of the Decision Tree model.
7	<code>from sklearn.metrics import classification_report</code>	Displays precision, recall and F1-score of the model.
8	<code>from sklearn.metrics import confusion_matrix</code>	Creates a confusion matrix showing correct and incorrect predictions.
9	<code>pd.read_csv(r"F:\Downloads\Most Runs - 2018.csv")</code>	Loads the IPL 2018 Most Runs dataset from the CSV file.
10	<code>df.head()</code>	Displays the first five records of the dataset.

MVLU COLLEGE
A.I. Practical No.3

11	<code>df.shape</code>	Displays the number of rows and columns in the dataset.
12	<code>df.drop("Player", axis=1)</code>	Removes the Player name column because it is not used as a numerical feature.
13	<code>df.drop("POS", axis=1)</code>	Removes the position/rank column from the input features.
14	<code>df["HS"].str.replace("*", "")</code>	Removes the * symbol from the Highest Score (HS) values.
15	<code>pd.to_numeric()</code>	Converts dataset columns into numerical values required by the machine-learning model.
16	<code>df.dropna()</code>	Removes rows containing missing values.
17	<code>df["Runs"].median()</code>	Calculates the median number of runs.
18	<code>df["Performance"]</code>	Creates a new target column for player performance.
19	"High" if $x \geq \text{median}$ else "Low"	Classifies players as High or Low performance based on median runs.
20	<code>X = df.drop(["Performance", "Runs"], axis=1)</code>	Stores the input features while removing the target and Runs column.
21	<code>y = df["Performance"]</code>	Stores High/Low Performance as the target variable.
22	<code>train_test_split()</code>	Divides the dataset into 80% training and 20% testing data.
23	<code>DecisionTreeClassifier(criterion="entropy")</code>	Creates the Decision Tree model using entropy as the splitting criterion.
24	<code>model.fit(X_train, y_train)</code>	Trains the Decision Tree using the training data.

MVLU COLLEGE
A.I. Practical No.3

25	<code>model.predict(X_test)</code>	Predicts the performance category of the test data.
26	<code>accuracy_score(y_test, y_pred)</code>	Calculates the overall prediction accuracy of the model.
27	<code>classification_report()</code>	Evaluates precision, recall and F1-score for High and Low performance.
28	<code>confusion_matrix()</code>	Displays the number of correct and incorrect High/Low classifications.
29	<code>plot_tree()</code>	Generates the graphical representation of the trained Decision Tree.
30	<code>plt.title()</code>	Adds a title to the Decision Tree visualization.
31	<code>plt.show()</code>	Displays the Decision Tree graph on the screen.

Visualization / Output Table

Visualization / Output	Purpose	What It Shows
Dataset Table	Understand the dataset	IPL player batting statistics
Dataset Shape	Understand dataset size	Number of rows and columns
Median Runs	Create classification target	Median value used for High/Low classification
Performance Distribution	Understand classes	Number of High and Low performance players
Training/Test Split	Prepare data for ML	80% training and 20% testing data
Accuracy	Measure overall performance	Percentage of correctly classified players
Classification Report	Detailed evaluation	Precision, Recall and F1-score

MVLU COLLEGE
A.I. Practical No.3

Confusion Matrix	Analyze predictions	Correct and incorrect High/Low predictions
Decision Tree	Visualize the model	Features, conditions and predicted performance classes

Code:

```

Practical 3.py - D:\Haripr\AI\Practical 3.py (3.13.3)
File Edit Format Run Options Window Help
print("Hariprasad vishwakarma T127")
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Load the dataset
df = pd.read_csv(r"F:\Downloads\Most Runs - 2018.csv")
print("First 5 rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
# Remove unnecessary columns
if "Player" in df.columns:
    df = df.drop("Player", axis=1)
if "POS" in df.columns:
    df = df.drop("POS", axis=1)
# Clean Highest Score column
if "HS" in df.columns:
    df["HS"] = df["HS"].astype(str).str.replace("+", "", regex=False)
    df["HS"] = pd.to_numeric(df["HS"], errors="coerce")
# Convert columns to numeric
for column in df.columns:
    df[column] = pd.to_numeric(df[column], errors="coerce")
# Remove missing values
df = df.dropna()
# Create Performance category
median_runs = df["Runs"].median()
print("\nMedian Runs:", median_runs)
df["Performance"] = df["Runs"].apply(
    lambda x: "High" if x >= median_runs else "Low"
)
print("\nPerformance Distribution:")
print(df["Performance"].value_counts())
# Separate input and output
# Remove Runs because Performance was created from Runs
X = df.drop(["Performance", "Runs"], axis=1)
y = df["Performance"]
print("\nNumber of Features:", X.shape[1])
print("Number of Samples:", X.shape[0])

```

MVLU COLLEGE

A.I. Practical No.3

```
Practical 3.py - D:\Hari\AI\Practical 3.py (3.13.5)
File Edit Format Run Options Window Help
print(df["Performance"].value_counts())
# Separate input and output
# Remove Runs because Performance was created from Runs
X = df.drop(["Performance", "Runs"], axis=1)
y = df["Performance"]
print("\nNumber of Features:", X.shape[1])
print("Number of Samples:", X.shape[0])
# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

# Create Decision Tree
model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=4,
    random_state=42
)

# Train the model
model.fit(X_train, y_train)
print("\nDecision Tree trained successfully!")
# Prediction
y_pred = model.predict(X_test)
print("\nPredicted Values:")
print(y_pred)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Low", "High"]
    )
)

# Confusion Matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
# Decision Tree Visualization
plt.figure(figsize=(20, 10))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=["Low", "High"],
    filled=True,
    rounded=True
)
plt.title("Decision Tree - Cricket Player Performance Classification")
plt.show()
```

Output:

Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE

A.I. Practical No.3

```
"IDLE Shell 3.13.5"
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: D:/Hari/A.I/Practical 3.py =====
hariprasad vishwakarma T127
First 5 rows:
   POS  Player  Mat  Inns  NO  Runs  ...  BF      SR  100  50  4s  6s
0    1  Kane Williamson  17   17   3   735  ...  516  142.44  0   8   64  28
1    2    Rishabh Pant  14   14   1   684  ...  394  173.60  1   5   68  37
2    3         KL Rahul  14   14   2   659  ...  416  158.41  0   6   66  32
3    4    Ambati Rayudu  16   16   2   602  ...  402  149.75  1   3   53  34
4    5    Shane Watson  15   15   1   555  ...  359  154.59  2   2   44  35

[5 rows x 14 columns]

Dataset Shape:
(138, 14)

Median Runs: 58.0

Performance Distribution:
Performance
High      69
Low       69
Name: count, dtype: int64

Number of Features: 11
Number of Samples: 138

Training Samples: 110
Testing Samples: 28

Decision Tree trained successfully!

Predicted Values:
['Low' 'Low' 'High' 'Low' 'High' 'Low' 'High' 'High' 'High' 'Low' 'High'
 'High' 'High' 'High' 'High' 'Low' 'Low' 'High' 'High' 'Low' 'Low' 'High'
 'Low' 'Low' 'Low' 'Low' 'Low' 'Low']

Accuracy: 0.9642857142857143
Accuracy Percentage: 96.42857142857143 %

Dataset Shape:
(138, 14)

Median Runs: 58.0

Performance Distribution:
Performance
High      69
Low       69
Name: count, dtype: int64

Number of Features: 11
Number of Samples: 138

Training Samples: 110
Testing Samples: 28

Decision Tree trained successfully!

Predicted Values:
['Low' 'Low' 'High' 'Low' 'High' 'Low' 'High' 'High' 'High' 'Low' 'High'
 'High' 'High' 'High' 'High' 'Low' 'Low' 'High' 'High' 'Low' 'Low' 'High'
 'Low' 'Low' 'Low' 'Low' 'Low' 'Low']

Accuracy: 0.9642857142857143
Accuracy Percentage: 96.42857142857143 %

Classification Report:
      precision    recall  f1-score   support

     Low         1.00      0.93      0.96         14
     High         0.93      1.00      0.97         14

   accuracy          0.96         28
  macro avg          0.97      0.96      0.96         28
 weighted avg          0.97      0.96      0.96         28

Confusion Matrix:
[[13  1]
 [ 0 14]]
```

Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE

A.I. Practical No.3

Figure 1

